JOURNAL OF COMBINATORIAL THEORY, Series A 50, 285–300 (1989)

On Arithmetical Algorithms over Finite Fields

DAVID G. CANTOR*

University of California at Los Angeles, Los Angeles, California 90024

Communicated by the Managing Editors

Received September 27, 1987

Standard methods for calculating over $GF(p^n)$, the finite field of p^n elements, require an irreducible polynomial of degree n with coefficients in $GF(p)$. Such a polynomial is usually obtained by choosing it randomly and then verifying that it is irreducible, using a probabilistic algorithm. If it is not, the procedure is repeated. Here we give an explicit basis, with multiplication table, for the fields $GF(p^{p^k})$, for $k = 0, 1, 2,...$, and their union. This leads to efficient computational methods, not requiring the preliminary calculation of irreducible polynomials over finite fields and, at the same time, yields a simple recursive formula for irreducible polynomials which generate the fields. The fast Fourier transform (FFT) is a method for efficiently evaluating (or interpolating) a polynomial of degree $<n$ at all of the nth roots of unity, i.e., on the finite multiplicative subgroups of F, in $O(n \log n)$ operations in the underlying field. We give an analogue of the fast Fourier transform which efficiently evaluates a polynomial on some of the additive subgroups of F. This yields new "fast" algorithms for polynomial computation. © 1989 Academic Press, Inc.

INTRODUCTION

Standard methods for calculating over $GF(p^n)$, the finite field of p^n elements, require an irreducible polynomial of degree n with coefficients in $GF(p)$. Such a polynomial is usually obtained by choosing it randomly and then verifying that it is irreducible, using a probabilistic algorithm. If it is not, the procedure is repeated (see, e.g., [5 or 7, Section 4.6.2]). Here we give an explicit basis, with multiplication table, for the fields $GF(p^{p^k})$, for $k = 0, 1, 2,...$, and their union. This leads to efficient computational methods, not requiring the preliminary calculation of irreducible polynomials over finite fields and, at the same time, yields a simple recursive formula for irreducible polynomials which generate the fields.

Besides its importance in scientific and engineering applications, the fast Fourier transform (FFT) is the fundamental method underlying "fast" multiplication and a number of other modern, "fast" algorithms [2, 7]. It

* This research was supported in part by the NSA.

0097-3165/89 $3.00
Copyright © 1989 by Academic Press, Inc.
All rights of reproduction in any form reserved.

provides an efficient method for evaluating (or interpolating) a polynomial of degree $<n$ at the n nth roots of unity (when they exist) in $O(n \log n)$ operations in the underlying field F. To use the FFT effectively, n must be a prroduct of a number of small factors. Indeed, if the factorization

$$n = n_1 n_2 n_3 \cdots n_r$$

is used, then the amount of work required is proportional to

$$n(n_1 + n_2 + \cdots + n_r).$$

When the n_i are chosen "small," e.g., each $n_i = 2$ or 4, then the above $O(n \log n)$ bound is attained. Unfortunately, if the underlying field has finite characteristic, then it may not contain n nth roots of unity for any integer n which has only "small" factors. For example, over the two-element field, one is restricted to roots of unity of odd order, and then to obtain, for example, a primitive root of unity of order $n = 3^m$, one must work in an extension field of degree $\geqslant 2 \cdot 3^{m-1}$. This substantially increases the work and destroys many of the advantages of the FFT. In this situation the actual bit-complexity of the FFT is not at all clear.

Some of the difficulties that arise in practice, for example, in the multiplication of two polynomials of degree n over a finite field of characteristic p, are described by Wagh and Morgera [11] who are primarily interested in minimizing the number of multiplications. See also [7]. Schönhage [10] has given a recursive algorithm for multiplying two polynomials of degree n which has bit complexity $O(n \log n \log \log n)$. However, his algorithm does not provide a method for evaluating or interpolating polynomials.

The FFT is used for evaluating a polynomial at the nth roots of unity; these roots form a multiplicative subgroup of the underlying field F. Here we give an analogue which will evaluate a polynomial on an additive subgroup of (a finite algebraic extension of) the field F. There will be one such subgroup, of p^m elements, for $m = 1, 2, 3, \ldots$.

When used for evaluating a polynomial on the subspace of $n = p^m$ elements, the algorithm requires $O(n \log n)$ multiplications and

$$O(n(\log n)^{1 + \log_p((p+1)/2)}) = O(n(\log n)^2)$$

"scalar" operations. (Here and throughout, unless otherwise stated, the constants implied by a "O" may depend upon p).

When used for multiplying two polynomials of degree $<n$ over a finite field of p elements, the actual bit complexity is also

$$O(n(\log n)^{1 + \log_p((p+1)/2)}).$$

As usual, to form the product of two polynomials f and g, each of degree

$\leqslant r$, we evaluate each of them at $\geqslant 2r+1$ points α obtaining the values $f(\alpha)$ and $g(\alpha)$, then we form the products $h(a)=f(\alpha)\,g(\alpha)$, and finally we compute the (unique) interpolating polynomial of degree $\leqslant 2r$ which passes through the points $(\alpha, h(\alpha))$; see Winograd [12, 13] for a discussion of this general method which, under certain assumptions, he shows to be optimal.

1. Finite Fields

Suppose that $F=F_0$ is a finite field of characteristic p and that F_k is the smallest extension field of F_0 containing $GF(p^{p^k})$; F_k is, of course, the splitting field of the polynomial

$$t^{p^{p^k}}-t$$

over F. We may assume that the F_k are chosen so that

$$F_0 \subset F_1 \subset F_2 \subset \cdots.$$

Then $\dim_F(F_k/F_{k-1}) \leqslant p$ and $\tilde{F}=\bigcup F_k$ is a field; it is the splitting field of the sequence of polynomials

$$t^{p^{p^k}}-t, \qquad k=1, 2, 3,...$$

over F. Note that if $F=GF(p^r)$ and $(p, r)=1$, then $\dim_F(F_k/F_{k-1})=p$ and hence $\dim_F(F_k/F)=p^k$.

Define the linear transformation $S\colon \tilde{F} \to \tilde{F}$ by $St=t^p-t$ and define the polynomial

$$s(t)=t^p-t=\prod_{\alpha \in GF(p)} (t-\alpha).$$

Define polynomials $s_0(t)=t$ and inductively

$$s_{m+1}(t)=s(s_m(t)), \qquad m=0, 1, 2,....$$

Thus, if $x \in \tilde{F}$, then $S^m x=s_m(x)$. By induction we obtain

$$s_m(t)=\sum_{i=0}^{m} (-1)^{m-i}\binom{m}{i} t^{p^i} \qquad \text{and} \qquad t^{p^m}=\sum_{i=0}^{m}\binom{m}{i} s_i(t)$$

(here $\binom{m}{i}$ denotes the ordinary binomial coefficient reduced (mod p)). In particular, when m is a power of p, say $m=p^k$, then

$$s_{p^k}(t)=t^{p^{p^k}}-t \qquad \text{and} \qquad t^{p^{p^k}}=s_{p^k}(t)+s_0(t).$$

Thus $s_m(t)$ has degree p^m and has $\leqslant(m+1)$ non-zero coefficients. It is a "linearized" polynomial; this means that $s_m(x+y)=s_m(x)+s_m(y)$. Equivalently, it means that only exponents which are powers of p appear explicitly in $s_m(t)$; i.e., $s_m(t)$ can be written in the form $\sum_j a_j t^{p^j}$. See [9] for more details. Define

$$W_m=\{x\in\tilde{F}\mid S^m x=0\}=\{x\in\tilde{F}\mid s_m(x)=0\}.$$

Then W_m is a vector subspace of $\tilde{F}$ over $GF(p)$. Since W_1, the kernel of S, has dimension 1 over $GF(p)$, $\dim_{GF(p)}(W_m/W_{m-1})\leqslant 1$ and hence $\dim_{GF(p)}(W_m)\leqslant m$. However, when m is a power of p, say $m=p^k$, then $W_{p^k}=GF(p^{p^k})$, hence has dimension $m=p^k$ over $GF(p)$. It follows that $\dim_{GF(p)}(W_m)=m$ for $m=1, 2, 3,\ldots$ and that S maps W_m onto W_{m-1}.

We now give explicit, computationally useful, bases and multiplication tables for the infinite field $\tilde{F}$. We assume for the remainder of this section that $F=F_0=GF(p)$ (it is easy to see that the bases constructed here will also serve as bases when $F=GF(p^r)$, where $(r,p)=1$, and that a slight modification of them will work when $p\mid r$).

Suppose then that $\{u_0, u_1, u_2,\ldots\}$ is a sequence of elements from the algebraic closure of $GF(p)$ satisfying

$$u_j^p-u_j=(u_0u_1\cdots u_{j-1})^{p-1}+[\text{a sum of monomials of lower degree}], \tag{1.1}$$

where by [a sum of monomials of lower degree] we mean a sum of the form

$$\sum_{\mathbf{i}} c_i u_0^{i_0}u_1^{i_1}\cdots u_{j-1}^{i_{j-1}},$$

with the c_i in $GF(p)$ and where the sum is over all vectors $\mathbf{i}=(i_0, i_1, \ldots, i_{j-1})$ for which all of the components i_l are $\leqslant(p-1)$, and at least one is $<(p-1)$ (when $j=0$ we require that $u_0^p-u_0=1$). Suppose m is a positive integer whose expansion to the base p is $m=m_km_{k-1}\cdots m_0$, so that $m=\sum_{i=0}^k m_ip^i$ and $0\leqslant m_i\leqslant p-1$ for $0\leqslant i\leqslant k$ with $m_k\neq 0$. Define $\gamma_m=m_{i_0}$, where i_0 is the smallest integer with $m_{i_0}\neq 0$; i.e., $m_0=m_1=\cdots=m_{i_0-1}=0$, but $m_{i_0}\neq 0$.

Next define $y_0=1$, and, for $m\geqslant 1$, with base p expansion as above, define

$$y_m=u_0^{m_0}u_1^{m_1}\cdots u_k^{m_k} \tag{1.2}$$

so that, in particular, $y_{p^r}=u_r$ for $r=0, 1, 2, \ldots$

Repeated applications, as necessary, of Eqs. (1.1) allow us to write any monomial of the form $\prod_{j=0}^k u_j^{i_j}$ as a linear combination, with coefficients in

$GF(p)$, of monomials of the same form, but satisfying, in addition, $i_j \leqslant p-1$ for $0 \leqslant j \leqslant k$. It follows, in particular, that any product $y_m y_n$ can be written as a linear combination of the y_i of the form $\sum_{i=0}^{j} a_{mni} y_i$ with all a_{mni} in $GF(p)$ and with $j \leqslant m+n$. Moreover, if m and n are both $<p^k$, then we may choose $j<p^k$ (i.e., $a_{mni}=0$ when $i \geqslant p^k$). More generally, if $m<rp^k$ and $n<sp^k$, then we may choose $j<(r+s-1)p^k$. Finally, suppose that when m and n are added in the base p, there are no "carries." Then $y_{m+n}=y_m y_n$.

It will follow from the next theorem that the a_{mni} give the multiplication table for the field $\tilde{F}$.

THEOREM 1.1. *The sequence* $\{y_0, y_1, y_2, \ldots\}$ *is a basis for* $\tilde{F}$. *Indeed,* $\{y_0, y_1, \ldots, y_m\}$ *is a basis for* W_{m+1} *and* y_m *is in* $W_{m+1}-W_m$. *If* $m \geqslant 1$, *then* $Sy_m - \gamma_m y_{m-1} \in W_{m-1}$.

Proof. We prove the last statement by induction on m. Specifically, we assume that if $1 \leqslant l < m$, then $Sy_l = \gamma_l y_{l-1} + \delta_{l-2}$, where $\delta_{l-2} \in W_{l-1}$. If $m=1$, then

$$Sy_1 = Su_0 = u_0^p - u_0 = 1 = \gamma_1 y_0.$$

Now suppose that $m>1$ and that $P=p^k$ is the largest power of p which is $\leqslant m$. We can write $m=aP+b$, where $1 \leqslant a \leqslant p-1$ and $0 \leqslant b \leqslant P-1$. It is immediate from (1.2) that $y_m = y_P^a y_b$. Note that for any u, v we have

$$\begin{aligned} S(uv) &= (uv)^p - uv = (u^p - u)\, v^p + u(v^p - v) \\ &= (Su)\, v^p + uSv \\ &= (Su)(Sv+v) + uSv. \end{aligned}$$

There are 3 cases:

1. If $a=1$ and $b=0$, then $\gamma_m=1$ and the result is a restatement of (1.1), for the [sum of monomials of lower degree] is an element of W_{P-1}.

2. If $a>1$ and $b=0$, then $\gamma_m=a$ and, by (1.2), $y_m = y_P y_n$, where $n=(a-1)P$. Since $\gamma_P=1$, we obtain

$$\begin{aligned} Sy_m &= S(y_n y_P) \\ &= S(y_n)(Sy_P + y_P) + y_n S(y_P) \\ &= (\gamma_n y_{n-1} + \delta_{n-2})(y_{P-1} + \delta_{P-2} + y_P) + y_n(y_{P-1} + \delta_{P-2}) \\ &= \gamma_n y_{n-1} y_P + y_n y_{P-1} + \delta_{m-2}, \end{aligned}$$

where

$$\delta_{m-2} = (\gamma_n y_{n-1} + \delta_{n-2})(y_{P-1} + \delta_{P-2}) + \delta_{n-2} y_P + y_n \delta_{P-2}.$$

By the above remarks, $\delta_{m-2} \in W_{m-1}$ and, since there are no carries when the additions $n+(P-1)$ and $(n-1)+P$ are computed in the base p, $y_n y_{P-1} = y_{n-1} y_P = y_{m-1}$. Since $\gamma_n = a-1$, we obtain $Sy_m = ay_{m-1} + \delta_{m-2} = \gamma_m y_{m-1} + \delta_{m-2}$, with $\delta_{m-2} \in W_{m-1}$.

3. If $b \neq 0$, then $m = A + b$, where $A = ap^k$, and $\gamma_m = \gamma_b$. Then, as above,

$$\begin{aligned} Sy_m &= S(y_A y_b) \\ &= S(y_A)(Sy_b + y_b) + y_A S(y_b) \\ &= (\gamma_A y_{A-1} + \delta_{A-2})(\gamma_b y_{b-1} + \delta_{b-2} + \gamma_b) + y_A(\gamma_b y_{b-1} + \delta_{b-2}) \\ &= \gamma_b y_A y_{b-1} + \delta_{m-2}, \end{aligned}$$

where

$$\delta_{m-2} = (\gamma_A y_{A-1} + \delta_{A-2})(\gamma_b y_{b-1} + \delta_{b-2} + y_b) + y_A \delta_{b-2}.$$

It follows, as before, that $\delta_{m-2} \in W_{m-1}$ and that $y_A y_{b-1} = y_{m-1}$, hence that $Sy_m = \gamma_m y_{m-1} + \delta_{m-2}$.

This proves the last statement of the theorem. It is immediate that $Sy_m \in W_m - W_{m-1}$. Since S maps $W_{m+1} - W_m$ onto $W_m - W_{m-1}$, we find that $y_m \in W_{m+1} - W_m$ and hence that $\{y_0, y_1, ..., y_m\}$ is a basis for W_{m+1}. ∎

A special case of the $p=2$ case of this theorem is proved, using an entirely different method, by Conway in [6] as a part of his construction of an algebraically closed field of characteristic 2.

There are p^{p^j-1} choices for the "sum of monomials of lower degree" in 1.1, hence p^{p^j} choices for u_j and $(p-1)p^{p^j}$ choices for its non-zero scalar multiples. By the last theorem, u_j is an element of $W_{p^j+1} - W_{p^j}$. This latter set contains $p^{p^j+1} - p^{p^j} = (p-1)p^{p^j}$ elements. Thus every element of $W_{p^j+1} - W_{p^j}$ can be represented as a non-zero multiple of an appropriately defined u_j.

Since the elements $S^n y$ are polynomials in y, it follows, in particular, that $y_{p^{k+1}-1}$ is a generator of the field F_{k+1} over $GF(p)$. Now $y_{p^{k+1}-1}$ can be written as a polynomial in u_k, hence $F_{k+1} = F(u_k)$. A different proof of this latter result, in the special case that $Su_k = (u_0 u_1 \cdots u_{k-1})^{p-1}$, for $k = 1, 2, 3, ...$ is given by Adleman and Lenstra in [1].

An even simpler sequence is given by the condition that $Su_k = u_{k-1}^{2p-1}$. For then we have

THEOREM 1.2. *Suppose that* $Su_0 = 1$, *that* $Su_1 = u_0$ *if* $p = 2$, *and that otherwise* $Su_k = u_{k-1}^{2p-1}$. *Then the sequence* $\{u_k\}$ *satisfies condition* (1.1).

Proof. Induction on k. When $k=1$ and $p=2$ the result is clear. Otherwise we have

$$\begin{aligned}
Su_k &= u_{k-1}^p u_{k-1}^{p-1} \\
&= (Su_{k-1} + u_{k-1})\, u_{k-1}^{p-1} \\
&= u_{k-1}^{p-1}\, Su_{k-1} + u_{k-1}^p \\
&= u_{k-1}^{p-1}\, Su_{k-1} + (Su_{k-1} + u_{k-1}) \\
&= (u_{k-1}^{p-1} + 1)\, Su_{k-1} + u_{k-1} \\
&= (u_{k-1}^{p-1} + 1)((u_0 u_1 \cdots u_{k-2})^{p-1} \\
&\quad + [\text{a sum of monomials of lower degree}]) + u_{k-1} \\
&= (u_0 u_1 \cdots u_{k-1})^{p-1} + [\text{a sum of monomials of lower degree}],
\end{aligned}$$

where, when $k=1$, the empty product $(u_0 u_1 \cdots u_{k-2})$ is defined to be 1. ∎

Using this latter sequence $\{u_k\}$ we now give a simple recursive construction of a sequence of irreducible polynomials $f_k(X)$ in $GF(p)[X]$ satisfying $\deg(f_k(X)) = p^k$. Let ω be a primitive $(2p-1)$th root of unity in the algebraic closure of $GF(p)$. Define $f_1(X) = X^p - X - 1$. If $p=2$, put $f_2(X) = f_1(X^2 - X)$; otherwise put

$$g_k(X^{2p-1}) = \prod_{i=0}^{2p-2} f_{k-1}(\omega^i X)$$

$$f_k(x) = g_k(x^p - x).$$

It is clear, inductively, that $g_k(X)$ is a polynomial in $GF(p)[X]$ of degree p^{k-1}, that $f_k(X)$ is a polynomial in $GF(p)[X]$ of degree p^k, and that u_{k-1} is a root of it. Since $\deg(F_k : GF(p)) = p^k$, the polynomial $f_k(X)$ is irreducible over $GF(p)$.

2. Outline of the FFT Algorithm

We now consider the analogue of the FFT for additive subgroups. It is for evaluating a polynomial on one of the subspaces W_m, or more generally on one of its translates.

Since W_m/W_{m-1} has dimension 1 as a vector space over $GF(p)$, there exist p vectors $0 = v_0, v_1, ..., v_{p-1}$ in W_m such that W_m is the disjoint union of the cosets $v_i + W_{m-1}$, $0 \leqslant i < p$; the elements $S^{m-1} v_i$ are the p elements of $GF(p)$.

Now suppose that $a(t)$ is a polynomial of degree $< p^m$, with coefficients in F. The algorithm for evaluating $a(t)$ at all of the elements of the coset

$w+W_m$ may be described as follows: Compute p polynomials $b_0(t)$, $b_1(t),\ldots, b_{p-1}(t)$, such that $b_i(x)=a(x)$ for $x\in w+v_i+W_{m-1}$. Since $w+v_i+W_{m-1}$ is the set of zeros of the polynomial $s_{m-1}(t-w-v_i)$, we may obtain $b_i(t)$ as the remainder when $a(t)$ is divided by the sparse polynomial $s_{m-1}(t-w-v_i)=s_{m-1}(t)-s_{m-1}(w+v_i)$; $b_i(t)$ has degree $<p^{m-1}$. We now proceed recursively, and for each i satisfying $0\leqslant i<p$ evaluate the polynomial $b_i(t)$ on the coset $w+v_i+W_{m-1}$. The recursion stops when the $b_i(t)$ all have degree 0, i.e., are constants. These constants will be the values of the original polynomial evaluated at the elements of W_m.

Let c_m be one less than the number of non-zero coefficients of the polynomial $s_m(t)$, (i.e., c_m is the number of non-zero binomial coefficients of the form $\binom{m}{i}$ for $0\leqslant i\leqslant m-1$) and put $C_n=\sum_{m=0}^{n} c_m$. We will show later that

$$C_n \ll n^{1+\log_p((p+1)/2)}$$

where, as asual, the symbol $\ll$ means that the expression on its left is less than a constant times the expression on its right. (Here, and throughout, unless otherwise stated, this constant, as with O, may depend upon p.)

The polynomial

$$s_{m-1}(t-w-v_i)=s_{m-1}(t)-s_{m-1}(w+v_i)$$

has constant term $-s_{m-1}(w+v_i)$, has leading coefficient 1, and has c_{m-1} other non-zero coefficients (which are binomial coefficients reduced $(\bmod p)$). Dividing $a(t)$ by it requires no more than $\deg(a(t))\,c_{m-1}$ "multiply-adds" by a binomial coefficient reduced $(\bmod p)$ and $\deg(a(t))$ "multiply-adds" by the constant term $-s_{m-1}(w+v_i)$. In bounding the complexity of the algorithms described here, we shall call the first type of operation (multiplication by a reduced binomial coefficient followed by an addition) an "A-operation" and the second type (multiplication by an arbitrary element followed by an addition an "M-operation." The amount of work required for an A-operation is similar to that required for an addition in F, while the work for required for an M-operation is similar to that required for a multiplication in F. The computation of all p of the $b_i(t)$ requires $\leqslant pc_{m-1}\deg(a(t))\leqslant c_{m-1}p^{m+1}$ A-operations and $\leqslant p\deg(a(t))\leqslant p^{m+1}$ M-operations.

Suppose we start with a polynomial $a(t)$ of degree $<n=p^m$ and that we wish to evaluate it at each point of W_m. Using the above procedure we first obtain p polynomials, each of degree $<p^{m-1}$, defined on the p cosets of W_{m-1} in W_m; then we obtain p^2 polynomials, each of degree $<p^{m-2}$, defined on the p^2 cosets of W_{m-2} in W_m; $\cdots$; finally, we obtain p^m polynomials, each of degree $<p^0$ (i.e., constants), defined on the p^m

cosets of W_0 in W_m (i.e., the p^m elements of W_m). The total number of A-operations is no greater than

$$\begin{aligned} p^{m+1} \sum_{i=0}^{m-1} c_i &\ll p^{m+1} C_{m-1} \\ &\ll pn((\log_p n)^{1+\log_p((p+1)/2)}) \\ &\ll pn(\log_p n)^2 \end{aligned}$$

and the number of M-operations is no greater than $p^{m+1}m \ll pn \log_p n$.

Conversely, given p polynomials $b_0(t), b_1(t), b_2(t), \ldots, b_{p-1}(t)$ defined on the cosets $w + v_i + W_{m-1}$, we define the polynomial

$$a(t) = -\sum_{i=0}^{p-1} \frac{s_m(t-w)}{s_{m-1}(t-w-v_i)} b_i(t).$$

We will show, in the next section, that $a(x) = b_i(x)$ for $x \in w + v_i + W_{m-1}$. Applying this construction recursively, we may compute the coefficients of a polynomial with prescribed values on the set W_m. The estimate for the number of steps required in this construction is the same as in the previous construction.

3. Detailed Description of the Algorithm

Since $Sx = x^p - x$, we have

$$t^p - t = \prod_{\alpha \in GF(p)} (t - \alpha),$$

and

$$\begin{aligned} s_h(t) - Sx &= (s_{h-1}(t) - x)^p - (s_{h-1}(t) - x) \\ &= \prod_{\alpha \in GF(p)} (s_{h-1}(t) - x - \alpha). \end{aligned}$$

This yields the sequence of factorizations

$$\begin{aligned} s_m(t) &= \prod_{x_1 \in W_1} (s_{m-1}(t) - x_1) \\ &= \prod_{x_2 \in W_2} (s_{m-2}(t) - x_2) \\ &\vdots \\ &= \prod_{x_m \in W_m} (s_0(t) - x_m). \end{aligned}$$

Thus to evaluate the polynomial $a(t)$ of degree $<p^m$ at all points of W_m we divide $a(t)$ by each of the polynomials $s_{m-1}(t)-x_1$, where $x_1 \in W_1$, then divide each of the remainders by the appropriate one of the polynomials $s_{m-2}(t)-x_2$, where $x_2 \in W_2$ and $Sx_2 = x_1$, ..., and finally divide by the appropriate one of the polynomials $s_0(t)-x_m = t-x_m$, where $x_m \in W_m$, and $Sx_m = x_{m-1}$. Then the remainders after the last division are the desired functional values.

To describe this more explicitly, let us define a collection of polynomials

$$\mathscr{A}_m = \{a_x^{(j)}(t) \mid 0 \leqslant j \leqslant m,\ x \in W_{m-j}\},$$

with $\deg(a_x^{(j)}(t)) < p^j$. Specifically, we define $a_x^{(j)}$ to the unique polynomial of degree $<p^j$ such that $a_x^{(j)}(t) = a(t)$ for t satisfying $s_j(t) = x$; equivalently $a_x^{(j)}(t) \equiv a(t) \pmod{s_j(t)-x}$. In particular, $a_0^{(m)}(t) = a(t)$. Then, since $s_j(t)-x$ divides $s_{j+1}(t)-Sx$, we obtain for $x \in W_j$,

$$a_x^j(t) \equiv a_{Sx}^{(j+1)}(t) \pmod{s_j(t)-x}. \tag{3.1}$$

Hence, we obtain the sequence of equalities:

$$a(x) = a_{S^m x}^{(m)}(x) = a_{S^{m-1}x}^{(m-1)}(x) = \cdots = a_{S^1 x}^{(1)}(x) = a_x^{(0)}(x).$$

The polynomials $a_x^{(0)}(t)$ are constants $(=a_x^{(0)}(x))$, and hence computation of the polynomials $a_x^j(t)$, successively, for $j=m-1, j-2, ..., j=0$, suffices for computation of the values of the polynomial $a(x)$.

The algorithm thus consists of m steps, for $j=1, j=2, ..., j=m$, where the jth step consisting of the computation of each of the polynomials $a_x^{(m-j)}(t)$ as the remainder obtained by dividing the polynomial $a_{Sx}^{(m-j+1)}(t)$ by the polynomial $s_{m-j}(t)-x$. (Possible simplifications will be discussed later.)

The reverse problem of interpolation consists of starting with the polynomials of degree 0 (constants) $\{a_x^{(0)}(t) = a_x \in W_m\}$ and constructing a polynomial $a(t)$ such that $a(x) = a_x$ for $x \in W_m$). This is done by successively, for $j=1$, $j=2, ..., j=m$, defining, for each $y \in W_{m-j}$, the polynomial

$$a_y^{(j)}(t) = -\sum_x \frac{s_j(t)-y}{s_{j-1}(t)-x}\, a_x^{(j-1)}(t), \tag{3.2}$$

where the sum is over those $x \in W_{m-j+1}$ satisfying $Sx = y$. Suppose $y = S^j u$, where $u \in W_m$; then $S^{j-1}u \in W_{m-j+1}$. If $Sx = y$ then $S(S^{(j-1)}u - x) = y - y = 0$. Hence $s_{j-1}(u) - x \in W_1 = GF(p)$. Since s is linearized, if $Sx = y$, then

$$\begin{aligned} s_j(t) - y &= s(s_{j-1}(t) - x) \\ &= \prod_{\alpha \in GF(p)} (s_{j-1}(t) - x - \alpha). \end{aligned}$$

Hence,

$$\begin{aligned} a_y^{(j)}(t) &= -\sum_x \frac{a_x^{(j-1)}(t)}{s_{j-1}(t)-x}(s_j(t)-y) \\ &= -\sum_x \frac{a_x^{(j-1)}(t)}{s_{j-1}(t)-x} \prod_{\alpha \in GF(p)} (s_{j-1}(t)-x-\alpha) \\ &= -\sum_x a_x^{(j-1)}(t) \prod_{\alpha \in GF(p)^*} (s_{j-1}(t)-x-\alpha) \end{aligned}$$

(where as usual $GF(p)^*$ denotes the non-zero elements of $GF(p)$). Substituting u for t yields

$$a_y^{(j)}(u) = -\sum_x a_x^{(j-1)}(u) \prod_{\alpha \in GF(p)^*} (s_{j-1}(u)-x-\alpha).$$

The last product is 0 unless $s_{j-1}(u)=x$ in which case, by Wilson's theorem, it is

$$\prod_{\alpha \in GF(p)^*} (-\alpha) = -1,$$

and then $a_{Sx}^{(j)}(u) = a_y^{(j)}(u) = a_x^{(j-1)}(u)$. Hence,

$$a_0^{(m)}(u) = a_{S^m u}^{(m)}(u) = a_{S^{m-1}u}^{(m-1)}(u) = \cdots = a_{Su}^{(1)}(u) = a_u^{(0)}(u),$$

and $a_0^{(m)}(t)$ is the desired interpolating polynomial.

Summarizing we have

THEOREM 3.1. *Suppose we are given a polynomial $a(t) \in \tilde{F}(t)$ of degree $<p^m$. Define the family $\mathscr{A}_m$ by putting $a_0^{(m)}(t) = a(t)$ and using* (3.1) *to define the remaining $a_x^{(j)}(t)$. Then the polynomials $a_x^{(0)}(t)$ are constants and $a(x) = a_x^{(0)}(t)$ for $x \in W_m$. Conversely, suppose we are given $a_x \in \tilde{F}$ for $x \in W_m$. Define the family $\mathscr{A}_m$ by putting $a_x^{(0)}(t) = a_x$ for $x \in W_m$ and using* (3.2) *to define the remaining $a_k^{(j)}(t)$. Then $a_0^{(m)}(t)$ is a polynomial of degree $<p^n$ satisfying $a_0^{(m)}(x) = a_x$ for all $x \in W_m$.*

The computation of $a_{Sx}^{(j)}(t)$ from $a_x^{(j+1)}(t)$ requires the division of a polynomial of degree less than p^{j+1} by the polynomial $s_j(t)-x$ of degree p^j, which has the leading coefficient 1, c_j other non-zero coefficients (which are binomial coefficients reduced (mod p)), and one other (usually) non-zero coefficient x. Hence the computation requires at most $c_j(p-1)p^j$ A-operations and $(p-1)p^j$ M-operations. Since, for each j, there are p^{m-j} polynomials $a_x^{(j)}(t)$, computing all of them requires at most $c_j(p-1)p^m$ A-operations and $m(p-1)p^m$ M-operations.

The calculation (3.2) can be achieved by multiplying the polynomial $a_x^{(j-1)}$ by $s_j(t)-y$ and then dividing it by $s_{j-1}(t)-x$ for each x satisfying $Sx=y$. The multiplication requires $c_j p^{j-1}$ A-operations and p^{j-1} M-operations, while the p divisions require a total of $p^{j+1}c_{j-1}$ A-operations and p^{j+1} M-operations. The summations need not be counted for they can be incorporated into the A-operations. Thus the entire interpolation step requires at most $p^{m-1}C_m+p^{m+1}C_{m-1}$ A-operations and $m(p^2+1)p^{m-1}$ M-operations.

For implementation purposes the following representation of the elements of W_m is useful. Let $y_0, y_1, ..., y_{m-1}$ be a basis of W_m satisfying $Sy_i=y_{i-1}$ for $1\leqslant i\leqslant m-1$. Then every $x\in W_m$ can be written uniquely as a finite sum of the form $x=\sum_{i=0}^{m-1}\alpha_i y_i$, with all $\alpha_i\in GF(p)$. We may consider the α_i as integers in the range $0\leqslant\alpha_i<p$ and thus associate to x a unique integer

$$\alpha=\sum_{i=0}^{m-1}\alpha_i p^i. \tag{3.3}$$

Then for $j\leqslant m$, the subspace W_j corresponds to the non-negative integers $<p^j$, and if x corresponds to α, then Sx corresponds to $\lfloor\alpha/p\rfloor$.

If α is a non-negative integer, we may write it in the form (3.3) and denote by x_α the element $\sum_i\alpha_i y_i$ in F which corresponds to it. Using this notation we can describe the algorithm as follows.

Evaluation. We assume given a polynomial $a(t)$ of degree $<p^m$ with coefficients in $\tilde{F}$ which we wish to evaluate at all points of W_m:

> Put $b_0^{(m)}(t)=a(t)$. Then, successively, for $j=1, j=2, ..., j=m$, define, for $0\leqslant k<p^j$, the polynomial $b_k^{(m-j)}(t)$ as the remainder when $b_{\lfloor k/p\rfloor}^{(m-j+1)}(t)$ is divided by $s^{(m-j)}(t)-x_k$.

Then the polynomials $b_k^{(0)}(t)$ are constants and are the desired values. That is, $a(x_k)=b_k^{(0)}(t)$.

Interpolation. We assume given values $b_k\in\tilde{F}$ for $0\leqslant k<p^m$ and we wish to find a polynomial $a(t)$ such that $a(x_k)=b_k$ for all such k:

> Put $b_k^0(t)=b_k$. Then, successively, for $j=1$, $j=2, ..., j=m$, define, for $0\leqslant k<p^{m-j}$, the polynomial
>
> $$b_k^j(t)=-\sum_{l=0}^{p-1}b_{pk+l}^{(j-1)}(t)\frac{s_j(t)-x_k}{s_{j-1}(t)-x_{pk+l}}.$$

Then $a(t)=b_0^{(m)}(t)$ is the desired polynomial.

The following formulas may be used to simplify and reduce the complexity of the division and interpolation steps of the algorithm:

Suppose $f=f(t)$ is a polynomial of degree $<p^m$ and $s=s_{m-1}(t)$ is a polynomial of degree p^{m-1}. In the evaluation algorithm we wish to obtain the remainders of f when it is divided by polynomials of the form $s-\alpha_i$, $0\leqslant i<p$. In the interpolation algorithm we are given the remainders and we wish to recover f.

To this purpose, put $f_0=f_0(t)=f$ and successively for $j=0$, $j=1,\ldots,$ $j=p-1$, define polynomials $f_{j+1}=f_{j+1}(t)$ and $u_{j+1}=u_{j+1}(t)$ by the division

$$f_j=(s-\alpha_j)f_{j+1}+u_{j+1},$$

where $\deg(u_j)<p^{m-1}$. Then, inductively if $f_j\neq 0$,

$$\deg(f_j)=\deg(f_{j-1})-p^{m-1}<p^m-jp^{m-1}=(p-j)\,p^{m-1}.$$

Define $Q_{jk}=R_{jk}=0$ for $j<k$ and for $0\leqslant j\leqslant k<p$ define

$$Q_{jk}=f_{j+1}-(\alpha_j-\alpha_k)\,Q_{j+1,k},\ R_{jk}=u_{j+1}-(\alpha_j-\alpha_k)\,R_{j+1,k}. \tag{3.4}$$

Then we have

THEOREM 3.2. *If $j\leqslant k$, then $f_j=(s-\alpha_k)\,Q_{jk}+R_{jk}$. In particular, substituting $j=0$ shows that the remainder when $f=f_0$ is divided by $s-\alpha_k$ is R_{0k}.*

Proof. When $j=k$, we have $Q_{jj}=f_{j+1}$ and $R_{jj}=u_{j+1}$, hence $f_j=(s-\alpha_j)\,Q_{jj}+R_{jj}$. For $j<k$ we proceed by induction as j decreases. We obtain

$$\begin{aligned}
f_j&=(s-\alpha_j)f_{j+1}+u_{j+1}\\
&=(s-\alpha_k)f_{j+1}-(\alpha_j-\alpha_k)f_{j+1}+u_{j+1}\\
&=(s-\alpha_k)f_{j+1}-(\alpha_j-\alpha_k)((s-\alpha_k)\,Q_{j+1,k}+R_{j+1,k})+u_{j+1}\\
&=(s-\alpha_k)(f_{j+1}-(\alpha_j-\alpha_k)\,Q_{j+1,k})+(u_{j+1}-(\alpha_j-\alpha_k)\,R_{j+1,k})\\
&=(s-\alpha_k)\,Q_{jk}+R_{jk}.
\end{aligned}$$

Starting with $f=f_0$, one can compute the f_i and the u_j; then one can recursively compute the desired remainders R_{0k} (note that it is not necessary to compute the Q_{jk}). Assuming that the differences $\alpha_j-\alpha_k$ are precomputed, the computation of f_i and u_i requires $(p^m-ip^{m-1})\,c_m$ A-operations and (p^m-ip^{m-1}) M-operations. Hence the computation of all of the f_i requires $p^m(p-1)\,c_m/2$ A-operations and $p^m(p-1)/2$ M-operations. Computing the remainders R_{0k} from the u_i requires no A-operations and $p^m(p-1)/2$ M-operations. All told, $p^m(p-1)\,c_m/2$ A-operations and $p^m(p-1)$ M-operations are required. If we are

evaluating a polynomial of degree $p^n - 1$, then the above computation must be performed p^{n-m} times for each value of m satisfying $0 \leqslant m < n$, yielding a total of $p^n(p-1)\, C_{n-1}/2$ A-operations and $np^n(p-1)$ M-operations. The method described earlier in this section requires the same number of M-operations and (roughly) twice as many A-operations.

Conversely, if one is given the remainders R_{0k}, the entire process can be reversed. First $u_1 = R_{00}$ is known. From (3.4) we obtain the formula $R_{j+1,k} = (u_{j+1} - R_{jk})/(\alpha_j - \alpha_k)$. For $j=0$, $k=1$ it enables computation of $R_{11} = u_2$. Proceeding, and using the same formula for $k=2$ allows computation of R_{12}, and then computation of $R_{22} = u_3$. Continuing in the same way allows computation of all of the u_j. Then the f_j can be computed in reverse order. The total amount of work is the same as in the evaluation scheme, above (under the assumption that either the elements $1/(\alpha_j - \alpha_k)$ are precomputed or else that a division is to be counted the same as a multiplication).

4. Polynomial Multiplication

Suppose $a(t)$ and $b(t)$ are polynomials in $\tilde{F}[t]$ of degree $n < p^m/2$. Then $c(t) = a(t)\, b(t)$ can be calculated by (1) evaluating $a(\alpha)$ and $b(\alpha)$ at all points of W_m, (2) putting $c(\alpha) = a(\alpha)\, b(\alpha)$, and (3) then interpolating to obtain the coefficients of $c(t)$. This requires $O(m^{1+\log_p((p+1)/2)}\, p^m)$ A-operations and $O(mp^m)$ M-operations.

Suppose that $m < p^k$ and that the field F_k has basis $1, \theta, \theta^2, ..., \theta^{k^k-1}$. Put $r = \lfloor (p^k - 1/2 \rfloor$ and suppose $a(t)$ and $b(t)$ are two polynomials in $F[t] = \mathrm{G}F(p)[t]$, each of degree $\leqslant r$. Then $c(t) = a(t)\, b(t)$ has degree $\leqslant 2r \leqslant p^k - 1$. Hence $c(\theta) = a(\theta)\, b(\theta)$ uniquely determines $c(t)$, and the product $a(t)\, b(t)$ can be computed by one multiplication in the field F_k.

We now consider computation with polynomials whose coefficients are in $\mathrm{G}F(p)$, and we assume for the remainder of this section that $F = \mathrm{G}F(p)$. Any polynomial in $\mathrm{G}F(p)[t]$ of degree $< nr$ can be written in the form $\sum_{i=0}^{n-1} a_i(t)\, T^i$, where $a_i(t)$ is a polynomial in t of degree $< r$ and $T = t^r$. To multiply such polynomials, it suffices to multiply polynomials of the form $\sum_{i=0}^{n-1} a_i(\theta)\, T^i$ of degree $< n$ and with coefficients in F_k. Thus if $n < p^m/2$, where $m = p^k$ and $r = \lfloor (p^k - 1)/2 \rfloor$, then two such polynomials can be multiplied in $O(nm^{1+\log_p((p+1)/2)})$ A-operations and $O(nm)$ M-operations with elements in the field F_k. In the field F_k, addition has bit complexity $O(p^k)$ and multiplication has bit complexity $O(k^2 p^k)$ the latter follows, for example, from recursive use of the method we are describing). It follows that the bit complexity, using this method, of multiplication of two polynomials of degree N in $\mathrm{G}F(p)[t]$ is $O(N(\log N)^{1+\log_p((p+1)/2)})$ as $N \to \infty$.

5. Binomial Coefficients

We now indicate how to obtain the estimate for C_m used earlier. The binomial coefficient $\binom{m}{j}$ is non-zero (mod p) if, and only if, in the addition, expressed in base p, of $j+(m-j)=m$ there are no "carries." Suppose that, when written to the base p, m has the expansion $m=m_{k-1}m_{k-2}\cdots m_0$ and j has the expansion $j=j_{k-1}j_{k-2}\cdots j_0$. There will be no carries if, and only if, $j_i \leqslant m_i$ for $0 \leqslant i \leqslant k-1$. It follows that the number of non-zero binomial coefficients is the number of m-tuples $j=j_{k-1}j_{k-2}\cdots j_0$ satisfying $0 \leqslant j_i \leqslant m_i$ for $0 \leqslant i \leqslant k-1$, hence that $c_m+1=\sum_{i=0}^{k-1}(1+m_i)$ and

$$C_{p^k}+(p^k+1)=\sum_{m=0}^{p^k}(c_m+1)=\sum_{\mathbf{m}}\prod_{0\leqslant i<k}(1+m_i),$$

where the second sum is over all vectors $\mathbf{m}=(m_{k-1}, m_{k-2}, ..., m_0)$ satisfying $0 \leqslant m_i < p$ for $0 \leqslant i < k$. Hence

$$C_{p^k}+(p^k+1)=\prod_{i=0}^{k-1}\sum_{m=1}^{p} m=(p(p+1)/2)^k.$$

Thus, if m is a power of p, then

$$C_m=m^{1+\log_p((p+1)/2)}$$

and, for all m,

$$C_m=O(m^{1+\log_p((p+1)/2)}).$$

References

1. Adleman and Lenstra, Finding irreducible polynomials over finite fields, *in* "Proceedings, Eighteenth Annual Symposium on Theory of Computing, Berkeley, CA," pp. 350–355, Assoc. Comput Mach. New York, 1986.
2. A. V. Aho, J. E. Hopcroft, and J. D. Ullman, "The Design and Analysis of Computer Algorithms," Addison–Wesley, Reading, MA, 1974.
3. A. V. Aho, K. Steiglitz, and J. D. Ullman, Evaluating polynomials at fixed sets of points, *SIAM J. Comput.* **14** (1975), 533–539.
4. C. de Boor, FFT as nested multiplication with a twist, *SIAM J. Sci. Statist. Comput.* **1** (1980), 173–178.
5. D. G. Cantor and H. Zassenhaus, A new algorithm for factoring polynomials over finite fields, *Math. Comp.* **36** (1981), 587–592.
6. J. H. Conway, "On Numbers and Games," Academic Press, New York, 1976.
7. D. E. Knuth, "The Art of Computer Programming, 2nd ed. Vol. II, "Seminumerical Algorithms," pp. 420–441, Addison–Wesley, Reading, MA, 1981.

8. A. LEMPEL, G. SEROUSSI, AND S. WINOGRAD, On the complexity of multiplication in Finite Fields, *Theoret. Comput. Sci.* **22** (1983), 85–296.
9. R. LIDL AND H. NIEDERREITER, "Finite Fields," Addison–Wesley, Reading, MA.
10. A. SCHÖNHAGE, Schnelle Multiplikation von Polynomen über Körpern der Charakteristik 2, *Acta Inform.* **7** (1978), 395–398.
11. M. D. WAGH AND S. D. MORGERA, A new structured design method for convolutions over finite fields, Part I, *IEEE Trans. Inform. Theory* **IM-29** (1983), 583–595.
12. S. WINOGRAD, On computing the discrete Fourier transform, *Math. Comp.* **32** (1978), 175–199.
13. S. WINOGRAD, "Arithmetic Complexity of Computations," Soc. Indus. Appl. Math., Philadelphia, 1980.